Verified answers from component datasheets

AUTHOR
Shlok Bhusal

DATE
November 2026

SOURCE
github.com/Protonicwave/
datasheet-question-answering-assistant

Abstract A language model asked about a sensor datasheet can state a wrong value with nothing to show which value is wrong. This assistant answers from passages retrieved from six Bosch Sensortec datasheets and shows an answer only when code confirms that each quoted passage appears word for word on the cited page and that every number in the answer appears in a quote. A rejected answer is sent back to the model once with the reason. On 50 held-out questions, 40 of them answerable, the model alone answered 26.0 correctly per run and stated a wrong value for 4.3. With retrieval it answered 33.3 correctly, with one wrong answer in three runs, and that answer traces to a datasheet that gives two values for one quantity. The check alone withheld 7.3 correct answers per run; with one revision the assistant answered 32.7 correctly with 4.0 answerable questions declined, against 3.3 for unchecked retrieval. The check does not make answers more accurate. It guarantees that every number shown can be found in a passage on the page cited.

1 Introduction

An engineer bringing up a sensor reads its register settings, timing and supply limits from a datasheet of 59 to 162 pages. The six Bosch Sensortec datasheets used here, for the BME280 [1], BMP390 [2], BMI270 [3], BMM350 [4], BMA456 [5] and BHI360 [6], run to 597 pages between them. Asking a language model is faster than searching them, but the answer comes from what the model remembers. On the held-out questions of Section 3, a model answering from memory stated a wrong value for 4.3 of 50 questions per run, among them a reset value and a gyroscope sensitivity, and nothing in those answers told the reader which ones to distrust.

The usual remedy is retrieval: search the documents and give the model the passages it needs, so that it answers from text rather than memory [7]. Retrieval removes most wrong values, as Section 4 shows, but the reader still takes the model's word that the passage says what the answer says. The model may cite a page and quote a passage that the page does not contain, or quote the right passage and state a number that is not in it. Checking either by hand means opening the datasheet, which is the work the assistant was meant to save.

The assistant described here closes that gap in code. Each answer must cite the page and quote the passage it came from. Before the answer is shown, code confirms that the quote appears word for word on the cited page and that every number in the answer appears in a quote. An answer that fails is sent back to the model once with the reason; if the revision also fails, the user is told there is no verified answer. The guarantee is deliberately narrow. Every number shown appears in a passage quoted verbatim from the page cited, so a reader can confirm it in seconds. It does not promise that the number answers the question.

The question this report answers is what the check costs and what it buys. Section 2 describes the system and Section 3 the evaluation: four arms, from the model alone to search with the check and one revision, scored by code on a development set and a held-out set. Section 4 gives the results, Section 5 interprets them and sets out what the check cannot catch, and Section A lists the commands and commits that reproduce each run.

2 System

Each question passes through retrieval, an answer with citations, the check and at most one revision, and ends in one of three outcomes. Figure 1 shows the path; this section takes each step in turn.

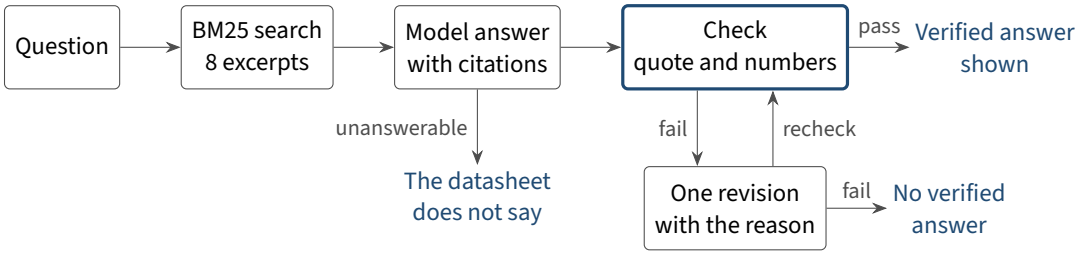


Figure 1 The path of one question. Only an answer that passes the check is shown. A rejected answer is sent back once with the reason it failed, and a revision that also fails is withheld.

2.1 Pages and text

The datasheets are downloaded from Bosch Sensortec and checked against a SHA-256 hash recorded in the repository, so every run reads the same files; the PDFs themselves are not redistributed. The text of each page is extracted with pdfium. Table rows come out in reading order, one row to a line, but the column a value sat in is lost: a row of minimum, typical and maximum values reads '`1.8 2.8 µA`' with no sign of which column was empty. Figures and graphs carry no text layer and are invisible to the assistant.

One normaliser prepares text for both search and the check, so the two never disagree about what a page says. It applies Unicode compatibility folding, maps the Unicode minus sign and every dash to a hyphen, unifies the two micro signs, removes a hyphen between two letters, and collapses runs of white space. The hyphen rule exists because pdfium marks a line-break hyphen with one code point whether the word was split ('consist-ing') or genuinely hyphenated ('low-power'); removing every hyphen between letters on both sides of the comparison makes both spellings match.

2.2 Retrieval

Each page is cut into chunks of whole lines, merged up to about 300 words and never spanning two pages, and each chunk keeps its datasheet and page. Search ranks chunks with BM25 [8], with $k_1 = 1.2$ and $b = 0.75$, and passes the top eight, about 2,400 words, to the model. BM25 was chosen over embeddings because datasheet questions turn on exact terms such as register names, and because it is deterministic. When a question names a part, only that part's datasheet is searched. Without this rule a BMP390 question ranked BMA456 and BMI270 pages first: every page header repeats the part number, so as a search term it barely separates one datasheet from another. A question that shares no term with any chunk is declared unanswerable without a model call.

2.3 Answer

The model, `claude-sonnet-5-5`, receives the excerpts, each marked with its datasheet and page, and returns a structured reply: whether the excerpts answer the question, a short answer, and one to three citations, each naming a datasheet and page and quoting a passage of at least four words. The prompt asks for numbers written as the datasheet writes them, no unit conversions, and no number repeated from the question unless a quote contains it. A request the API refuses or truncates, a failed request, or a reply that does not match the schema is recorded as an error, never as a sign that the datasheet does not say.

2.4 The check

The check is a pure function of the answer and the page texts. An answer passes only if it cites at least one passage; every citation names a datasheet and page that exist; every quote is at least four words long and appears, after normalisation, on the page it cites; and every number in the answer appears in at least one of its quotes. The four-word minimum stops a quote such as '3.6 V', which appears on many pages, from passing as evidence. The numbers are pooled across citations because one answer may combine values from up to three passages.

Numbers are extracted as signed decimals, scientific notation and hexadecimal (`0x76`). A digit attached to letters, as in `BME280` or `I2C`, is not a number, and an `x` between digits separates two values, so `2.5x3.0` yields 2.5 and 3.0. Decimals compare by value with their sign, so 3.30 matches 3.3 but −40 does not match 40; hexadecimal compares as text, ignoring case.

2.5 Revision and outcomes

A rejected answer is sent back to the model in the same conversation, followed by the reason it failed, for example 'answer states 0, 2, which no quote contains'. The model is asked to fix the cause, either by quoting a passage that holds the number or by leaving out a number the question did not ask for, and is told not to drop the value the question asks for. One revision is allowed. The revision faces the same check, so it can move an answer from withheld to shown only by passing.

Each question ends in one of three outcomes. A verified answer is shown with its citations. 'No verified answer' means every draft failed the check: the datasheet may still state the value, and the reason is shown. 'The datasheet does not say' means the model found no passage that answers. A web page shows each verified answer beside the rendered datasheet page with the quoted passage highlighted, and marks a number's place on the page when the reader points at it in the answer.

3 Evaluation

Four arms answer the same questions on the six datasheets, three runs each, and code scores every answer against the values the datasheet states. The design aims to make each arm differ from the last in one step only, and to keep the questions used to build the system apart from the questions that measure it.

3.1 Arms

Arm A is the model alone: the same model and output schema as the assistant, with a prompt that supplies no datasheet text and asks for no citations. Arm B is the model answering from the eight retrieved excerpts, with its answer shown unchecked. Arm C passes each of B's stored answers through the check and withholds those that fail; it makes no model request of its own, so the check is the only difference between B and C. Arm D starts from the same stored answer and, when the check rejects it, asks for one revision as in Section 2. D therefore differs from C only by the revision, and from B by the check and the revision together.

Each arm ran three times on each question set, with up to four requests in flight. Three runs show how far the counts move from run to run; they are too few for a confidence interval, and the tables give the lowest and highest run instead.

3.2 Questions

A question names one datasheet and states whether that datasheet answers it. An answerable question lists the numbers and keywords a correct answer must state and the pages that state them, and may list alternative sets for the same value in other units, such as 0.5 hPa for 50 Pa. When the question file is loaded, code confirms that every expected number of the first set appears on one of the listed pages, so a mistyped value fails before any run.

The development set holds 100 questions, 80 answerable and 20 not, between 15 and 18 per datasheet. Candidates were drafted with Claude, and I checked each answer and page against the PDF. The answers to these questions were read while the system was built: the withheld answers of arm C on this set are what led to the revision step. Its figures therefore describe the system it shaped and are not the result.

The held-out set holds 50 further questions on the same six datasheets, 40 answerable and 10 not, the same proportion as the development set. It was written after the system was finished. Three Claude agents drafted 60 candidates, each with the verbatim passage that answers it, and were shown the development questions so as not to repeat them, but nothing about the revision step or its failures. Claude then removed nine doubtful candidates (an

ambiguous value, an unclear table column, a question too close to a development one, a value the scorer could not read) and checked the remaining flagged ones against the rendered pages. I approved that selection without checking each question by hand, so the held-out set was drafted and checked by Claude, not by me. The set was committed before it was first run, the code was frozen at that commit, and the set was run once; the results are reported unchanged.

3.3 Scoring

Code scores every answer; no model judges another. An answer to an answerable question is *correct* if it states every number and keyword of any one accepted set, keywords matching as whole words so that 'Pa' is not found in 'hPa'. It is *wrong* if it states numbers but none from any accepted set, and any answer to an unanswerable question is wrong. Declining an answerable question, whether the model says the datasheet does not say or the check withholds the answer, is a *false refusal*; declining an unanswerable one is a *correct refusal*. Everything else is *unplaced* and left for a person to judge: an answer that states some expected values but not all, or no numbers at all.

4 Results

Table 1 gives the held-out counts for every arm and Figure 2 plots the three that separate them.

The model alone answered 26.0 of the 40 answerable questions correctly and stated a wrong value for 4.3 questions per run, between 3 and 7. Its 13 wrong answers over three runs fell on seven questions. Two were wrong in every run, the BMA456 `INIT_CTRL` reset value and the BMA456 sensor time increment discussed below, and two in two runs, the BMA456 filter bandwidth at an output data rate of 100 Hz and the BHI360 gyroscope sensitivity. Seven of the 13 answers said they came from memory or were uncertain; all 13 stated a value.

With search, the model answered 33.3 correctly, refused all ten unanswerable questions in every run and gave one wrong answer in three runs. Every correct answer cited a page the question lists as stating the value. The three unplaced answers per run are correct when read by hand: each writes the unit against the number, as in '1.25ms', '12.5MHz' and '0.4mm', and the scorer's whole-word match does not find 'ms' in '1.25ms'. The same three answers are unplaced in arms C and D.

The check alone withheld 22 of B's answers over the three runs, 7.3 per run, and every one was to an answerable question. Twelve quoted fewer than four words. Ten stated a number that no quote contained: in six the number was part of register notation, such as 'bits 3..1' or `int_latch='0'`, and in the rest an operating condition, a table number or a page number mentioned in passing. C's correct count fell to 26.0 and its false refusals rose from B's 3.3 to 10.7. The check removed none of B's wrong answers: the one wrong answer passed it.

With one revision, D revised all 22 withheld answers and showed 20, each scored correct. The two it still withheld were the same question in two runs: the answer repeated the field name `t_sb[2:0]` from the question, and the check read 2 and 0 as numbers no quote contained. D answered 32.7 correctly with 4.0 false refusals, 0.6 and 0.7 from B's figures.

The single wrong answer of arms B, C and D is the same answer in run 2 to the question of how often the BMA456 `SENSORTIME_0` register increments. Page 16 of the datasheet says the register increments every 39.25 µs, and the

Table 1 Held-out set: 50 questions, 40 answerable, three runs per arm. Each count is the mean over the runs, with the lowest and highest run in brackets where they differ. No arm returned an API error.

Arm	Correct		Wrong		Correct refusal		False refusal		Unplaced	
A: model alone	26.0		4.3	(3 to 7)	9.7	(9 to 10)	9.7	(8 to 11)	0.3	(0 to 1)
B: model with search	33.3	(33 to 34)	0.3	(0 to 1)	10.0		3.3	(3 to 4)	3.0	
C: search with check	26.0	(25 to 27)	0.3	(0 to 1)	10.0		10.7	(9 to 12)	3.0	
D: search, check and retry	32.7	(32 to 34)	0.3	(0 to 1)	10.0		4.0	(3 to 5)	3.0	

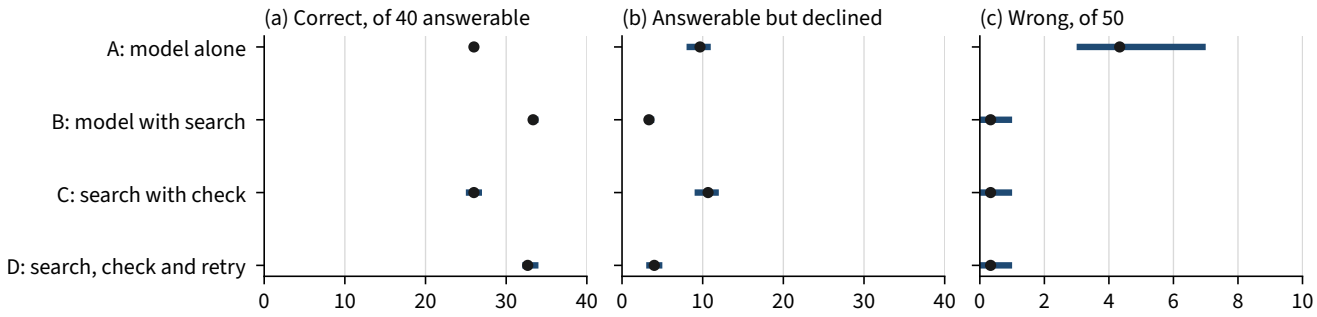


Figure 2 Held-out results per arm. Each dot is the mean of three runs and the bar spans the lowest to the highest run. Panel (c) counts all 50 questions, since answering an unanswerable question is also wrong.

question expects that value; pages 21 and 51 give 39.0625 µs [5]. In runs 1 and 3 the answer quoted both pages and stated both values, which the scorer accepts. In run 2 it quoted only pages 21 and 51 and gave 39.0625 µs, a value the datasheet does state. The model alone gave 39.0625 µs in all three runs. The datasheet contradicts itself, so these answers are scored wrong as committed but the fault lies in the question.

Table 2 gives the development figures. They follow the same pattern with larger gaps: the check alone cost B 24.3 correct answers per run, and the revision recovered all but 0.3 of them. In the development run all 73 revisions passed the check at the first attempt, so a second revision was never requested; the limit was then lowered from two revisions to one, which leaves these figures unchanged and cuts the worst case from three requests per question to two.

5 Discussion and limits

Search, not the check, removed the wrong answers. The model alone gave a wrong value for 4.3 held-out questions per run; with eight retrieved excerpts it gave one wrong answer in three runs, and that answer quoted a real passage, so the check passed it. On these questions the check did not make the shown answers more accurate.

What the check bought is a property of every answer shown: each number in it appears in a passage quoted word for word from the page it cites. A reader who doubts a value opens one page, finds a highlighted passage and confirms the number there, rather than searching the datasheet. Without the check, arm B's answers carry the same citations, but nothing confirms that the quote is on the page or that the number is in the quote. On this set B's citations happened to hold up; the check is what makes that true of every answer rather than of those examined.

The check alone paid for this property with recall: it withheld 7.3 of B's 33.3 correct held-out answers per run. Its rejections were mostly the model's own habits rather than unsupported claims. A three-word quote such as '"110": 10 ms' is the right evidence and simply too short for the rule; 'bits 3..1' is register notation the model wrote in its own words. Because these rejections are systematic, sending the reason back works: one revision recovered 20 of the 22 withheld answers and brought arm D to within 0.6 correct answers and 0.7 false refusals of B. The two answers

Table 2 Development set: 100 questions, 80 answerable, three runs per arm. These questions were read while the revision step was designed, so the figures show why it exists and are not the result. No arm returned an API error.

Arm	Correct		Wrong		Correct refusal		False refusal		Unplaced	
A: model alone	59.7	(58 to 62)	11.0	(9 to 13)	14.3	(13 to 15)	15.0	(12 to 17)	0.0	
B: model with search	77.0		0.0		20.0		3.0		0.0	
C: search with check	52.7	(52 to 54)	0.0		20.0		27.3	(26 to 28)	0.0	
D: search, check and retry	76.7	(76 to 77)	0.0		20.0		3.0		0.3	(0 to 1)

it could not recover came from one case the check gets wrong. A field name taken from the question, `t_sb[2:0]`, contains digits, and the check counts them as numbers the quote must contain. Exempting text copied from the question would fix this case, but it would also let an answer repeat a wrong value that the question itself contained.

5.1 What the check does not catch

The check confirms that a number is in a quote and the quote is on a page. It does not confirm that the quote answers the question. An answer can quote the wrong row, the wrong condition or the wrong column of a table and pass, and the BMA456 sensor time question shows that a datasheet can state two values for one quantity. A revision can also pass by dropping context: in the development run, one revision dropped the $\pm 8\ g$ range from an answer about the accelerometer's reset range, and it was still scored correct. The revision prompt forbids dropping the value the question asks for, but nothing in code enforces it, and a revision could in principle swap the requested value for another number from the same quote. The scorer would count a swapped value as wrong, but it cannot see dropped context, and the assistant prevents neither.

5.2 What the assistant cannot read

The assistant reads only the text layer. Graphs, timing diagrams and package drawings are invisible to it, and formulas extract as garbled text. In tables the column a value sat in is lost, so a row of minimum, typical and maximum values cannot tell the model which is which; two of the withheld held-out answers said as much. The number extractor reads '10,000' as 10 and 0, so a value written with a thousands separator can be neither verified nor scored, and one held-out candidate was dropped for that reason. The check compares numbers as written, so an answer converted from 1 mA to 1000 µA is withheld, and the prompt asks the model not to convert. Decimal commas and scanned datasheets without a text layer are not handled.

5.3 How far the result reaches

The held-out result rests on 50 questions, 40 of them answerable, on six datasheets from one manufacturer, answered by one model. The questions are on the same datasheets as the development set, so they test the system on new questions, not on new documents. The held-out set was drafted and checked by Claude, and Claude also answers the questions; a person checking each question against the PDF, as was done for the development set, would make the set independent of the model it measures. Three runs show how much the counts move between runs, at most four questions for any arm and category, but they are too few for an interval. The scorer undercounts correct answers when the unit is written against the number, which leaves three correct answers per run unplaced in arms B to D, and it marks a correct answer wrong when the question records only one of two values that the datasheet gives.

6 Conclusion

The aim was an assistant whose answers a reader can confirm without searching the datasheet. Code now checks every answer before it is shown: the quote must be on the cited page and every number must be in a quote. On 50 held-out questions, retrieval brought wrong answers from 4.3 per run to one in three runs, and that one came from a datasheet that states two values for the same quantity. The check alone withheld about a fifth of the correct answers; one revision with the reason for the rejection recovered 20 of 22, leaving the assistant 0.6 correct answers per run behind unchecked search, with every number it shows traceable to a quoted passage.

The check guarantees where a number came from, not that it answers the question. The open question is how often a verified answer quotes the wrong row, condition or column of a table. Answering it needs questions written by engineers who use these parts, checked against the PDF by a person, on datasheets the system has not seen, and a scorer that compares each answer with the table cell the question is about.

References

[1] *BME280 Data Sheet*, Bosch Sensortec (2024), document BST-BME280-DS001-24, revision 1.24.

[2] *BMP390 Datasheet*, Bosch Sensortec (2021), document BST-BMP390-DS002-07, revision 1.7.

[3] *BMI270 Datasheet*, Bosch Sensortec (2026), document BST-BMI270-DS000-08, revision 1.6.

[4] *BMM350: 3-axis magnetic sensor with high data rate*, Bosch Sensortec (2025), document BST-BMM350-DS001-27, revision 1.27.

[5] *BMA456 Data Sheet*, Bosch Sensortec (2026), document BST-BMA456-DS000-09, revision 3.6.

[6] *BHI360 Datasheet*, Bosch Sensortec (2026), document BST-BHI360-DS000-06, revision 1.9.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks (2020), arXiv:2005.11401 .

[8] S. Robertson and H. Zaragoza, Foundations and Trends in Information Retrieval 10.1561/1500000019 (2009).

A Reproducing the results

The repository's README gives installation. A run needs the datasheets ingested and an Anthropic API key in the environment variable `ANTHROPIC_API_KEY`. One command then produces each run:

```
uv run datasheet-assistant ingest
uv run datasheet-assistant evaluate --questions eval/held_out.jsonl --runs 3
```

Each run writes a new directory under `eval/runs/` holding every answer with its citations and revisions (`answers.jsonl`), the score of each question per arm and run (`scores.csv`) and the counts per arm (`summary.csv`, `summary.md`). Table 3 lists the committed runs. A rerun makes new model requests, so its counts will differ from these within roughly the spread the tables show.

Table 3 Committed evaluation runs. The held-out set was committed at `4b9dc7c` and run once at that commit.

Run directory	Question set	Arms	Results commit
`20261108T120946Z`	development	A to C	`be39273`
`20261108T125938Z`	development	A to D, up to two revisions	`91bd5e1`
`20261108T135144Z`	held-out	A to D, one revision	`164ba5b`

The figure in Section 4 is drawn from the held-out `summary.csv` by `docs/report/figures/results.py`:

```
uv run --group docs python docs/report/figures/results.py
```